

全同态加密实验报告

基于 Microsoft SEAL CKKS 的密文卷积实现 与旋转次数优化

作业 5: 4×4 输入与 3×3 卷积核的密文卷积

作业 6: 打包-旋转-累加策略与理论最小旋转次数

摘要

本实验使用开源全同态加密库 Microsoft SEAL 4.3.2, 在 CPU 环境下实现单输入单输出的密文卷积。输入为一个 4×4 单通道矩阵, 卷积核为 3×3 , 步长为 1, 不使用填充, 输出尺寸为 2×2 。实验采用 CKKS 近似同态加密方案, 将 16 个输入元素按行打包到一个密文的槽位中, 通过密文旋转、密文与明文逐槽相乘以及密文累加完成卷积。

作业 5 采用直接旋转累加法作为基线, 为卷积核 9 个位置分别构造槽位偏移, 除偏移 0 外共执行 8 次密文旋转。作业 6 在相同输入、参数和卷积定义下采用 Baby-Step Giant-Step (BSGS, 小步-大步) 策略, 将二维偏移 $4a + b$ 分解为水平小步和垂直大步, 复用中间旋转结果, 将旋转次数降低到 4 次。

固定样例中, 明文参考结果为 $[[348, 393], [528, 573]]$; 直接法最大绝对误差为 8.789110×10^{-6} , BSGS 法最大绝对误差为 4.853192×10^{-7} 。5 组随机测试全部通过。在当前运行环境中, 直接法平均耗时为 40.883 ms, BSGS 法平均耗时为 18.171 ms, 获得 2.250 倍加速。

理论分析进一步表明: 在单输入密文、按行槽位打包、一般稠密 3×3 卷积核以及由旋转、明文掩码乘法和密文加法构成的线性计算模型下, r 次旋转最多产生 2^r 种独立输入偏移。由于本实验需要 9 种偏移, 故 $2^3 < 9 \leq 2^4$, 至少需要 4 次旋转。BSGS 是达到该下界的一种具体构造, 而不是唯一构造; 其他不改变实验模型的方法最多与 4 次持平。

关键词: 全同态加密; Microsoft SEAL; CKKS; 密文卷积; 密文旋转; BSGS

目录

摘要 I

1 作业要求与实验目标 1

1.1 作业 5 要求 1

1.2 作业 6 要求 1

1.3 实验目标 1

2 全同态加密与 CKKS 原理 2

2.1 全同态加密的计算模型 2

2.2 CKKS 近似实数计算 2

2.3 SIMD 槽位打包 3

2.4 密文旋转与二维偏移映射 4

2.5 明文掩码的双重作用 5

3 全同态加密库的选择与配置 6

3.1 选择 Microsoft SEAL 6

3.2 选择 CKKS 的原因 6

3.3 参数配置与含义 7

4 实验环境、目录结构与复现方法 8

4.1 实验环境 8

4.2 工程目录结构 8

4.3 编译与运行 8

4.4 CMake 配置 9

5 作业 5: 直接密文卷积实现 9

5.1 明文参考卷积 9

5.2 固定实验数据 10

5.3 直接旋转累加法 10

5.4 直接法的优缺点 11

6 作业 6: BSGS 旋转复用实现 12

6.1 偏移分解 12

6.2 Baby-Step 阶段 12

6.3	分组明文乘加与掩码预放置	12
6.4	Giant-Step 阶段	13
7	正确性验证与性能测试	14
7.1	解密与输出槽提取	14
7.2	误差判定	15
7.3	随机测试	16
7.4	性能测试范围	16
8	实验结果	17
8.1	固定样例结果	17
8.2	随机测试结果	18
8.3	性能结果	18
8.4	核心运行日志	18
9	作业分析与理论最小旋转次数	19
9.1	作业 5 完成情况	19
9.2	作业 6 的优化效果	19
9.3	两级 BSGS 模型下的下界	19
9.4	一般旋转线性电路下的下界	20
9.5	BSGS 不是唯一的 4 次构造	20
9.6	Hoisting 和旋转密钥优化为何不改变逻辑次数	21
9.7	改变实验模型后能否少于 4 次	21
9.8	CKKS 误差分析	22
9.9	性能结果的解释与局限	22
10	结论	22
	参考资料	24
A	核心常量和数据布局	24

1 作业要求与实验目标

1.1 作业 5 要求

选择任一开源全同态加密库，使用单输入单输出的 4×4 输入和 3×3 卷积核，设置步长为 1、无填充，在输入保持加密的情况下实现卷积，并通过解密结果与明文结果对比验证正确性。

输出尺寸为

$$H_{\text{out}} = W_{\text{out}} = \frac{4 - 3}{1} + 1 = 2. \quad (1)$$

因此最终应得到一个 2×2 的密文输出。

1.2 作业 6 要求

在作业 5 基础上采用“打包 $\rightarrow$ 旋转 $\rightarrow$ 累加”策略，探索本次卷积实现中的旋转次数是否达到理论最小值。为保证比较公平，本实验设置如下对照组。

表 1 作业 5 与作业 6 的对照实验设计

对照项	作业 5：直接法	作业 6：BSGS 法
输入和卷积核	相同	相同
CKKS 参数与密钥	相同	相同
卷积定义	CNN 式互相关	CNN 式互相关
输入打包	单密文按行打包	单密文按行打包
旋转策略	每个偏移单独生成	小步和大步复用
预期旋转次数	8	4

1.3 实验目标

本实验需要完成以下目标：

1. 使用 Microsoft SEAL 的 CKKS 方案完成密文卷积；
2. 验证解密结果与明文卷积结果在误差阈值内一致；
3. 统计直接法和优化法的实际旋转调用次数；
4. 分析 BSGS 为何可以复用旋转结果；
5. 证明 4 次旋转是否为当前模型下的理论最小值；
6. 比较正确性、误差、旋转次数和运行时间。

2 全同态加密与 CKKS 原理

2.1 全同态加密的计算模型

全同态加密 (Fully Homomorphic Encryption, FHE) 允许计算方在不知道明文的情况下直接对密文执行运算。设明文为 x , 待计算函数为 f , 则理想关系为

$$\text{Dec}(\text{Eval}(f, \text{Enc}(x))) = f(x). \quad (2)$$

CKKS 属于近似同态加密, 其关系写为

$$\text{Dec}(\text{Eval}(f, \text{Enc}(x))) \approx f(x). \quad (3)$$

在本实验中, 输入矩阵由数据所有者编码和加密; 服务器端只使用公开的评估工具执行旋转、明文乘法、加法和重缩放; 最终由持有私钥的一方解密结果。

代码对应: 密钥和同态运算组件初始化。

```

1 KeyGenerator keygen(context);
2 const SecretKey secret_key = keygen.secret_key();
3
4 PublicKey public_key;
5 keygen.create_public_key(public_key);
6
7 GaloisKeys galois_keys;
8 keygen.create_galois_keys(galois_keys);
9
10 CKKSEncoder encoder(context);
11 Encryptor encryptor(context, public_key);
12 Evaluator evaluator(context);
13 Decryptor decryptor(context, secret_key);

```

Listing 1 密钥与 CKKS 运算组件初始化

其中, `PublicKey` 用于加密, `SecretKey` 用于解密, `GaloisKeys` 用于密文槽位旋转, `Evaluator` 负责执行同态运算。

2.2 CKKS 近似实数计算

CKKS 通过缩放因子将实数近似编码到多项式环中。本实验设置输入和明文掩码的初始缩放因子为

$$\Delta = 2^{40}. \quad (4)$$

密文与明文相乘后, 缩放因子近似变为

$$\Delta_{\text{mul}} = 2^{40} \times 2^{40} = 2^{80}. \quad (5)$$

由于 9 个乘积项具有相同层级和缩放尺度, 可以先完成累加, 再统一执行一次重缩放。

代码对应: 参数设置和统一重缩放。

```
1 EncryptionParameters parms(scheme_type::ckks);
2 constexpr std::size_t poly_modulus_degree = 8192;
3 parms.set_poly_modulus_degree(poly_modulus_degree);
4 parms.set_coeff_modulus(
5     seal::CoeffModulus::Create(
6         poly_modulus_degree, {60, 40, 60}));
7 constexpr double scale = static_cast<double>(1ULL << 40);
8
9 // Nine products are added first, then rescaled once.
10 evaluator.rescale_to_next_inplace(output.ciphertext);
```

Listing 2 CKKS 参数与重缩放

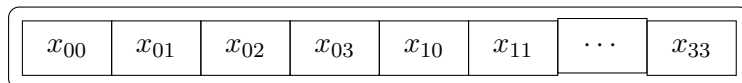
本实验没有密文与密文乘法, 因此不需要重线性化密钥; 乘法深度仅为 1, 也不需要 Bootstrapping。

2.3 SIMD 槽位打包

CKKS 支持 SIMD 式批量编码。程序将 4×4 输入按行展开为

$$[x_{00}, x_{01}, x_{02}, x_{03}, x_{10}, \dots, x_{33}], \quad (6)$$

并复制到 4096 个槽位向量的前 16 个位置, 其余位置补 0。随后对整个向量进行一次编码和加密。



一个 CKKS 密文中的前 16 个逻辑槽位

图 1 按行展开后的输入打包方式

一次旋转、乘法或加法会同时作用于所有槽位, 因此 4 个输出位置能够并行计算。

代码对应: 输入打包与加密。

```
1 Ciphertext encrypt_input(
```

```

2  const std::vector<double> &input,
3  CKKSEncoder &encoder,
4  Encryptor &encryptor,
5  double scale)
6  {
7      std::vector<double> packed(encoder.slot_count(), 0.0);
8      std::copy(input.begin(), input.end(), packed.begin());
9
10     Plaintext plain;
11     encoder.encode(packed, scale, plain);
12
13     Ciphertext encrypted;
14     encryptor.encrypt(plain, encrypted);
15     return encrypted;
16 }

```

Listing 3 输入打包函数 encrypt_input

2.4 密文旋转与二维偏移映射

输入每行有 4 个元素, 卷积核二维位置 (a, b) 在线性打包后对应偏移

$$d(a, b) = 4a + b, \quad a, b \in \{0, 1, 2\}. \quad (7)$$

因此 9 个卷积位置对应

$$D = \{0, 1, 2, 4, 5, 6, 8, 9, 10\}. \quad (8)$$

表 2 3×3 卷积核位置与线性旋转偏移

卷积核行	线性偏移	含义
第 1 行	0、1、2	当前行连续 3 个元素
第 2 行	4、5、6	下一行连续 3 个元素
第 3 行	8、9、10	下两行连续 3 个元素

由于不同接口对正负旋转方向的定义容易造成歧义, 程序先对探测向量 $[1, 2, 3]$ 旋转 1 个槽位并解密, 以自动判断 SEAL 正步长是否对应逻辑左旋。

代码对应：偏移表、方向检测和旋转计数。

```

1  const std::array<int, kKernelSize> kOffsets{
2      0, 1, 2, 4, 5, 6, 8, 9, 10};
3
4  void counted_rotate(
5      Evaluator &evaluator,
6      const Ciphertext &source,
7      int logical_left_step,
8      int rotation_sign,
9      const GaloisKeys &galois_keys,
10     Ciphertext &destination,
11     std::size_t &counter)
12 {
13     evaluator.rotate_vector(
14         source,
15         rotation_sign * logical_left_step,
16         galois_keys,
17         destination);
18     ++counter;
19 }
```

Listing 4 旋转偏移与计数封装

2.5 明文掩码的双重作用

CKKS 旋转是循环旋转，尾部数据可能绕回前部。为清除无效位置并乘上卷积核权重，需要构造明文掩码。本实验将 4 个输出保存在槽位

$$S = \{0, 1, 4, 5\}, \quad (9)$$

分别对应 $y_{00}, y_{01}, y_{10}, y_{11}$ 。

对于卷积核权重 k_i ，直接法掩码只在上述 4 个槽位写入 k_i ，其余槽位为 0。因此，明文掩码同时完成权重乘法和无效槽清零。

代码对应：直接法明文掩码。

```

1  const std::array<std::size_t, kOutputSize> kOutputSlots{
2      0, 1, 4, 5};
3
4  for (std::size_t i = 0; i < kKernelSize; ++i) {
```

```
5  std::vector<double> mask(slot_count, 0.0);
6  for (const std::size_t slot : kOutputSlots) {
7      mask[slot] = kernel[i];
8  }
9  encoder.encode(mask, scale, masks.direct[i]);
10 }
```

Listing 5 直接法掩码编码

3 全同态加密库的选择与配置

3.1 选择 Microsoft SEAL

本实验选择 Microsoft SEAL 4.3.2，主要原因如下：

1. 提供成熟的 C++ 接口和完整示例；
2. 支持 BFV、BGV 和 CKKS 方案；
3. 原生支持 SIMD 编码、密文旋转、明文乘法、密文加法和重缩放；
4. 支持通过 Galois Key 执行 CKKS 槽位旋转；
5. 能够在普通 CPU 环境中完成本作业规模的实验；
6. 使用 CMake 目标 SEAL::seal，便于构建和复现。

3.2 选择 CKKS 的原因

卷积输入和权重可以是浮点数，CKKS 原生支持近似实数计算和向量打包。与需先将实数缩放为整数的 BFV/BGV 方案相比，CKKS 更符合本实验的数值计算形式。

3.3 参数配置与含义

表 3 CKKS 和工程配置

参数	设置	作用
SEAL 版本	4.3.2	开源全同态加密库
方案	CKKS	近似实数向量计算
poly_modulus_degree	8192	提供 4096 个 CKKS 槽位
coeff_modulus	{60,40,60}	支持一次明文乘法和重缩放
初始 scale	2^{40}	控制近似编码精度
容许误差	10^{-4}	判断解密结果是否正确
编译标准	C++17	满足 SEAL 工程要求
编译优化	Release, -O3 -march=native	启用本机 CPU 优化

代码对应：参数有效性检查。

```
1 SEALContext context(parms);
2 if (!context.parameters_set()) {
3     throw std::runtime_error("invalid CKKS parameters");
4 }
```

Listing 6 SEALContext 参数检查

4 实验环境、目录结构与复现方法

4.1 实验环境

表 4 实验环境

项目	配置
操作系统	Ubuntu Linux
运行设备	CPU
编译工具	CMake + Ninja
C++ 标准	C++17
FHE 库	Microsoft SEAL 4.3.2
构建模式	Release

4.2 工程目录结构

```
1 fhe_conv_seal/
2 |-- CMakeLists.txt
3 |-- README.md
4 |-- install_and_run.sh
5 |-- run.sh
6 `-- src/
7     |-- main.cpp
```

Listing 7 实验工程目录

4.3 编译与运行

首次运行可使用一键脚本：

```
1 cd fhe_conv_seal
2 chmod +x install_and_run.sh run.sh
3 ./install_and_run.sh
```

Listing 8 一键安装和运行

SEAL 已经安装后，可直接执行：

```
1 cd fhe_conv_seal
```

```
2 ./run.sh
```

Listing 9 后续重复运行

4.4 CMake 配置

```
1 cmake_minimum_required(VERSION 3.22)
2 project(fhe_conv_seal LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 17)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6
7 find_package(SEAL 4.3 REQUIRED)
8 add_executable(fhe_conv src/main.cpp)
9 target_link_libraries(fhe_conv PRIVATE SEAL::seal)
10 target_compile_options(fhe_conv PRIVATE -O3 -march=native)
```

Listing 10 CMakeLists.txt 核心内容

5 作业 5: 直接密文卷积实现

5.1 明文参考卷积

本实验采用 CNN 中常见的 cross-correlation 形式, 即卷积核不做 180 度翻转:

$$y_{i,j} = \sum_{a=0}^2 \sum_{b=0}^2 x_{i+a,j+b} k_{a,b}. \quad (10)$$

明文函数用于产生统一参考结果, 同时作为直接法和 BSGS 法的正确性基准。

代码对应: 明文参考实现。

```
1 for (std::size_t out_r = 0; out_r < kOutputRows; ++out_r) {
2     for (std::size_t out_c = 0; out_c < kOutputCols; ++out_c) {
3         double sum = 0.0;
4         for (std::size_t kr = 0; kr < kKernelRows; ++kr) {
5             for (std::size_t kc = 0; kc < kKernelCols; ++kc) {
6                 const std::size_t input_index =
7                     (out_r + kr) * kInputCols + (out_c + kc);
8                 const std::size_t kernel_index =
9                     kr * kKernelCols + kc;
10                sum += input[input_index] * kernel[kernel_index];
```

```

11     }
12 }
13 output[out_r * kOutputCols + out_c] = sum;
14 }
15 }

```

Listing 11 plain_correlation 核心循环

5.2 固定实验数据

输入和卷积核分别为

$$X = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}. \quad (11)$$

卷积核 9 个权重均非零，可用于分析一般稠密 3×3 卷积的旋转下界。

代码对应：固定输入与卷积核。

```

1  const std::vector<double> input{
2      1, 2, 3, 4,
3      5, 6, 7, 8,
4      9, 10, 11, 12,
5      13, 14, 15, 16};
6
7  const std::vector<double> kernel{
8      1, 2, 3,
9      4, 5, 6,
10     7, 8, 9};

```

Listing 12 固定测试数据

5.3 直接旋转累加法

直接法为 9 个卷积核元素分别生成对应旋转视图，乘以对应的明文掩码后累加：

$$C_{\text{direct}} = \sum_{a=0}^2 \sum_{b=0}^2 P_{a,b} \odot \text{Rot}_{4a+b}(C_X). \quad (12)$$

偏移 0 直接使用原密文，其余 8 个非零偏移各执行一次旋转。因此直接法的操作数量为：8 次旋转、9 次密文与明文乘法、8 次密文加法和 1 次重缩放。

代码对应：direct_convolution。

```
1 for (std::size_t i = 0; i < kKernelSize; ++i) {
2     Ciphertext term;
3     if (kOffsets[i] == 0) {
4         term = encrypted_input;
5     } else {
6         counted_rotate(
7             evaluator,
8             encrypted_input,
9             kOffsets[i],
10            rotation_sign,
11            galois_keys,
12            term,
13            output.rotations);
14     }
15
16     evaluator.multiply_plain_inplace(term, masks.direct[i]);
17     if (!initialized) {
18         output.ciphertext = std::move(term);
19         initialized = true;
20     } else {
21         evaluator.add_inplace(output.ciphertext, term);
22     }
23 }
24 evaluator.rescale_to_next_inplace(output.ciphertext);
```

Listing 13 直接密文卷积

5.4 直接法的优缺点

直接法的优点是数据流清晰，每个卷积核位置与一个旋转偏移一一对应，容易实现和验证；缺点是没有利用偏移集合的二维结构，旋转结果之间无法复用，因此旋转次数较多。该方法适合作为作业 5 的正确性基线。

6 作业 6: BSGS 旋转复用实现

6.1 偏移分解

卷积偏移具有规则结构:

$$\{0, 1, 2, 4, 5, 6, 8, 9, 10\} = \{4a + b \mid a, b \in \{0, 1, 2\}\}. \quad (13)$$

BSGS 将完整偏移拆成两级:

- Baby-Step: 水平偏移 $b \in \{0, 1, 2\}$;
- Giant-Step: 垂直偏移 $4a \in \{0, 4, 8\}$ 。

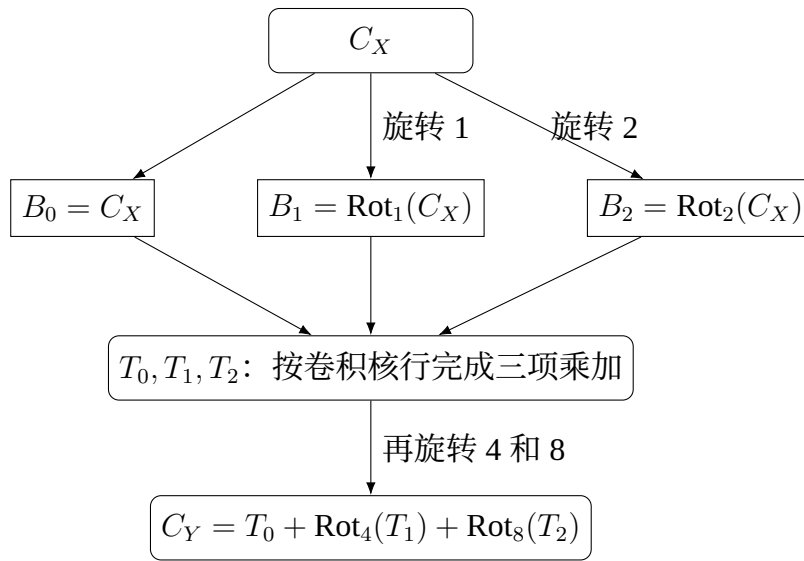


图 2 BSGS 密文卷积的数据流

6.2 Baby-Step 阶段

首先生成

$$B_0 = C_X, \quad B_1 = \text{Rot}_1(C_X), \quad B_2 = \text{Rot}_2(C_X). \quad (14)$$

偏移 0 不需要旋转, 因此该阶段只调用 2 次旋转。

6.3 分组明文乘加与掩码预放置

对卷积核每一行 a , 计算

$$T_a = \sum_{b=0}^2 Q_{a,b} \odot B_b. \quad (15)$$

由于 T_a 最终还要左旋 $4a$ 个槽位, BSGS 掩码应预先把有效数据放到

$$\text{pre_rotation_slot} = \text{output_slot} + 4a \quad (16)$$

的位置。该预移动发生在明文向量编码之前, 不增加密文旋转次数。

代码对应: BSGS 掩码预放置。

```

1 for (std::size_t a = 0; a < kKernelRows; ++a) {
2     for (std::size_t b = 0; b < kKernelCols; ++b) {
3         std::vector<double> mask(slot_count, 0.0);
4         for (const std::size_t output_slot : kOutputSlots) {
5             const std::size_t pre_rotation_slot =
6                 output_slot + 4 * a;
7             mask[pre_rotation_slot] =
8                 kernel[a * kKernelCols + b];
9         }
10        encoder.encode(mask, scale, masks.bsgs[a][b]);
11    }
12 }
```

Listing 14 BSGS 专用明文掩码

6.4 Giant-Step 阶段

完成三组乘加后, 执行

$$C_{\text{BSGS}} = T_0 + \text{Rot}_4(T_1) + \text{Rot}_8(T_2). \quad (17)$$

T_0 不旋转, 因此该阶段只使用 2 次旋转。总旋转次数为

$$2(\text{Baby-Step}) + 2(\text{Giant-Step}) = 4. \quad (18)$$

代码对应: bsgs_convolution。

```

1 std::array<Ciphertext, kKernelCols> baby;
2 baby[0] = encrypted_input;
3 counted_rotate(evaluator, encrypted_input, 1, rotation_sign,
4     galois_keys, baby[1], output.rotations);
5 counted_rotate(evaluator, encrypted_input, 2, rotation_sign,
6     galois_keys, baby[2], output.rotations);
7
```

```

8  std::array<Ciphertext, kKernelRows> groups;
9  for (std::size_t a = 0; a < kKernelRows; ++a) {
10     bool initialized = false;
11     for (std::size_t b = 0; b < kKernelCols; ++b) {
12         Ciphertext term = baby[b];
13         evaluator.multiply_plain_inplace(term, masks.bsgs[a][b]);
14         if (!initialized) {
15             groups[a] = std::move(term);
16             initialized = true;
17         } else {
18             evaluator.add_inplace(groups[a], term);
19         }
20     }
21 }
22
23 counted_rotate(evaluator, groups[1], 4, rotation_sign,
24               galois_keys, group1_rotated, output.rotations);
25 counted_rotate(evaluator, groups[2], 8, rotation_sign,
26               galois_keys, group2_rotated, output.rotations);
27
28 output.ciphertext = groups[0];
29 evaluator.add_inplace(output.ciphertext, group1_rotated);
30 evaluator.add_inplace(output.ciphertext, group2_rotated);
31 evaluator.rescale_to_next_inplace(output.ciphertext);

```

Listing 15 BSGS 密文卷积

7 正确性验证与性能测试

7.1 解密与输出槽提取

密文卷积完成后，程序使用私钥解密并解码全部槽位，最后提取槽位 0、1、4、5 组成 2×2 输出。

代码对应：解密与输出提取。

```

1  std::vector<double> decrypt_output(
2      const Ciphertext &ciphertext,
3      CKKSEncoder &encoder,
4      Decryptor &decryptor)

```

```

5 {
6     Plaintext plain;
7     decryptor.decrypt(ciphertext, plain);
8
9     std::vector<double> decoded;
10    encoder.decode(plain, decoded);
11    return extract_output(decoded);
12 }
13
14 for (const std::size_t slot : kOutputSlots) {
15     output.push_back(slots[slot]);
16 }

```

Listing 16 decrypt_output 与输出槽提取

7.2 误差判定

CKKS 是近似计算，使用最大绝对误差

$$E_{\max} = \max_i \left| y_i^{\text{FHE}} - y_i^{\text{plain}} \right| \quad (19)$$

进行验证，阈值设置为 10^{-4} 。固定样例同时检查：直接法旋转次数为 8、BSGS 法旋转次数为 4、两种密文结果均接近明文结果、两种密文方法之间相互一致。

代码对应：固定样例判定。

```

1 const double direct_error =
2     max_abs_error(expected, direct_values);
3 const double bsgs_error =
4     max_abs_error(expected, bsgs_values);
5 const double methods_error =
6     max_abs_error(direct_values, bsgs_values);
7
8 const bool fixed_pass =
9     direct.rotations == 8 &&
10    bsgs.rotations == 4 &&
11    direct_error < kTolerance &&
12    bsgs_error < kTolerance &&
13    methods_error < kTolerance;

```

Listing 17 误差计算和 PASS 条件

7.3 随机测试

程序使用固定随机种子生成 5 组 $[-1, 1]$ 范围内的输入和稠密卷积核。若卷积核权重绝对值小于 0.05，则调整为 ± 0.05 ，避免核元素接近 0 造成旋转需求退化。每组同时比较明文、直接法和 BSGS 法结果。

代码对应：随机数据生成。

```
1 std::mt19937_64 random_engine(20260727ULL);
2 std::uniform_real_distribution<double> distribution(-1.0, 1.0);
3
4 for (double &value : random_input) {
5     value = distribution(random_engine);
6 }
7 for (double &value : random_kernel) {
8     value = distribution(random_engine);
9     if (std::abs(value) < 0.05) {
10         value = value < 0.0 ? -0.05 : 0.05;
11     }
12 }
13
14 const bool pass =
15     random_direct.rotations == 8 &&
16     random_bsgs.rotations == 4 &&
17     error < kTolerance;
```

Listing 18 随机测试数据生成与判定

7.4 性能测试范围

计时只包含密文卷积，不包括密钥生成、输入编码、加密、掩码编码和解密。每种算法先执行一次不计时预热，再重复 5 次取平均值。

代码对应：benchmark_ms。

```
1 template <typename Function>
2 double benchmark_ms(Function &&function, std::size_t repeats)
3 {
4     function(); // Untimed warm-up.
5     const auto begin = std::chrono::steady_clock::now();
6     for (std::size_t i = 0; i < repeats; ++i) {
7         function();
```

```
8 }
9 const auto end = std::chrono::steady_clock::now();
10 const auto elapsed =
11     std::chrono::duration<double, std::milli>(end - begin).count();
12 return elapsed / static_cast<double>(repeats);
13 }
```

Listing 19 性能计时函数

8 实验结果

8.1 固定样例结果

明文卷积结果为

$$Y_{\text{plain}} = \begin{bmatrix} 348 & 393 \\ 528 & 573 \end{bmatrix}.$$

(20)

直接法和 BSGS 法解密结果如下。

表 5 固定样例结果

结果类型	2 × 2 结果
明文结果	[[348.000000, 393.000000], [528.000000, 573.000000]]
直接法	[[347.999991, 393.000001], [528.000000, 573.000000]]
BSGS 法	[[348.000000, 393.000000], [528.000000, 573.000000]]

表 6 固定样例验证指标

指标	直接法	BSGS 法
旋转次数	8	4
最大绝对误差	8.789110×10^{-6}	4.853192×10^{-7}
固定样例验证	PASS	PASS

两种密文方法之间的最大差异为 8.303791×10^{-6} ，同样小于 10^{-4} 。

8.2 随机测试结果

表 7 随机测试结果

测试编号	最大误差	结果
1	6.037814×10^{-7}	PASS
2	5.863962×10^{-7}	PASS
3	8.339157×10^{-7}	PASS
4	1.731711×10^{-7}	PASS
5	1.294341×10^{-6}	PASS

5 组随机测试全部通过，最坏误差为 1.294341×10^{-6} ，显著低于误差阈值。

8.3 性能结果

表 8 直接法与 BSGS 法性能比较

方法	平均耗时	旋转次数	相对速度
直接法	40.883 ms	8	1.000×
BSGS 法	18.171 ms	4	2.250×

加速比为

$$\text{Speedup} = \frac{40.883}{18.171} = 2.250.$$

(21)

BSGS 将旋转次数减少 50%，平均耗时降低约 55.56%。

8.4 核心运行日志

1

Fixed-case verification:

2

expected result = [[348, 393], [528, 573]]

3

direct rotations = 8 (expected 8)

4

BSGS rotations = 4 (expected 4)

5

direct max error = 8.789110e-06

6

BSGS max error = 4.853192e-07

7

method-to-method err = 8.303791e-06

8

fixed test = PASS

9

```

10 Random tests: 5/5 passed;
11 worst error = 1.294341e-06
12
13 Benchmark:
14   direct average = 40.883 ms
15   BSGS average  = 18.171 ms
16   speedup       = 2.250x
17
18 FINAL STATUS: PASS

```

Listing 20 程序核心输出

9 作业分析与理论最小旋转次数

9.1 作业 5 完成情况

作业 5 使用一个 CKKS 密文打包 16 个输入元素，通过直接偏移旋转、明文权重掩码乘法和密文累加完成卷积。固定样例、随机样例和误差验证均通过，证明程序能够在输入不解密的情况下得到正确的 2×2 输出。

直接法逻辑清晰，但没有复用偏移集合的二维结构，需要 8 次旋转。因此它适合作为基线，而不是最终的旋转最优实现。

9.2 作业 6 的优化效果

作业 6 与作业 5 计算完全相同的卷积，只改变旋转组织方式。BSGS 复用水平偏移 0、1、2，再按卷积核行执行垂直偏移 0、4、8，将旋转次数从 8 次降低为 4 次，同时保持 9 次明文乘法和 8 次加法不变。

因此，本次加速主要来自高成本旋转和密钥切换操作的减少，而不是改变卷积的数学定义或减少卷积核权重数量。

9.3 两级 BSGS 模型下的下界

设 Baby-Step 使用 r_b 次非零旋转，Giant-Step 使用 r_g 次非零旋转。加上两级各自不旋转的状态，能够形成的偏移组合数最多为

$$(r_b + 1)(r_g + 1). \quad (22)$$

稠密 3×3 卷积需要 9 个偏移，因此必须满足

$$(r_b + 1)(r_g + 1) \geq 9. \quad (23)$$

若总旋转数为 3, 最优整数分配为 $r_b = 1, r_g = 2$, 只能覆盖

$$(1 + 1)(2 + 1) = 6 < 9. \quad (24)$$

若总旋转数为 4, 取 $r_b = r_g = 2$, 可覆盖

$$(2 + 1)(2 + 1) = 9. \quad (25)$$

因此, 4 次是两级 BSGS 模型的最小值。

9.4 一般旋转线性电路下的下界

为了避免把“BSGS 构造”误认为“下界证明”, 进一步考虑更一般的单密文线性计算模型。假设允许任意顺序地执行:

- 密文旋转;
- 密文与任意明文掩码相乘;
- 密文加法;
- 不允许额外输入密文或加密前冗余移位副本。

开始时只有原始偏移集合 $S_0 = \{0\}$ 。明文乘法只改变系数和槽位选择, 不产生新的输入偏移; 加法只合并已有偏移。一次旋转可将某个已有偏移集合整体平移。设执行第 i 次旋转前, 所有中间密文涉及的偏移并集大小为 m_i , 一次新的旋转至多增加一个平移副本, 因此

$$m_{i+1} \leq 2m_i. \quad (26)$$

由 $m_0 = 1$ 递推得到

$$m_r \leq 2^r. \quad (27)$$

一般稠密 3×3 卷积需要 9 种独立偏移, 因此

$$2^r \geq 9 \implies r \geq \lceil \log_2 9 \rceil = 4. \quad (28)$$

这说明 4 次不仅是当前两级 BSGS 划分的最小值, 也是上述单密文旋转-掩码-加法模型中的一般下界。

9.5 BSGS 不是唯一的 4 次构造

BSGS 是达到 4 次下界的一种结构化实现, 但不是唯一实现。理论上, 二叉旋转树、分治式偏移生成或其他偏移基分解也可能用 4 次旋转覆盖 9 种偏移。然而, 由于一般下界已经是 4, 这些方法最多与 BSGS 持平, 无法在不改变实验模型的情况下做到 3 次或更少。

BSGS 的优势在于：

- 1. 与二维卷积核行列结构直接对应；
- 2. 掩码构造规则清晰；
- 3. 旋转步骤容易统计和复现；
- 4. 对更大的规则偏移集合可以自然推广。

9.6 Hoisting 和旋转密钥优化为何不改变逻辑次数

Hoisting 可以复用多个旋转中公共的密钥切换预处理，从而降低实际时间；只生成必要的 Galois Key 可以减少密钥存储。这些优化会改变底层成本和内存占用，但通常不会减少本实验定义的逻辑 rotate_vector 调用次数。

因此，报告应区分：

- **逻辑旋转次数：**本实验统计的 8 次与 4 次；
- **旋转的底层实现成本：**可通过 Hoisting、密钥选择和库内部优化进一步降低。

9.7 改变实验模型后能否少于 4 次

如果改变本作业的约束，服务器侧旋转次数可能低于 4，但相应代价必须同时报告。

表 9 可能减少服务器旋转次数的模型变化及代价

方法	可能效果	主要代价或变化
多个输入密文	可减少甚至消除部分旋转	加密次数、通信量和内存增加
冗余打包	客户端预先复制移位数据	将旋转工作转移到加密前，槽位利用率下降
稀疏卷积核	非零偏移数减少	不再代表一般稠密 3×3 卷积
可分离卷积核	拆为横向和纵向一维卷积	仅适用于具有低秩结构的卷积核
系数编码卷积	可能不使用槽位旋转	编码、输出布局和后处理模型发生变化
交互式协议	服务器操作可能减少	引入通信轮次，不再是纯非交互 FHE 评估

因此，本实验不能简单声称“任何 FHE 卷积都至少需要 4 次旋转”。严格结论是：在单输入密文、按行槽位打包、一般稠密 3×3 核以及旋转-掩码-加法模型下，4 次达到下界。

9.8 CKKS 误差分析

固定样例中 BSGS 误差小于直接法, 但这并不意味着 BSGS 在理论上必然更精确。两种方法的旋转顺序、加法结合顺序和中间密文噪声传播路径不同, 因此近似舍入误差可能略有差异。

应关注的是两种方法都满足

$$E_{\max} < 10^{-4}, \quad (29)$$

而不是依据某一次运行中误差较小来断言某种算法具有固定的精度优势。

9.9 性能结果的解释与局限

旋转次数降低 50%, 实际耗时降低约 55.56%, 说明旋转是当前程序的主要开销。密文旋转通常包含 Galois 自同构和密钥切换, 其成本明显高于普通密文加法。

不过 2.250 倍加速不应视为所有平台上的固定结论, 其数值会受到以下因素影响:

- CPU 型号、核心频率与缓存;
- SEAL 编译选项和底层 NTT 实现;
- 不同旋转步长所需的密钥和访问模式;
- 预热次数、重复次数和系统调度;
- 是否启用 Hoisting 或其他库内部优化。

本实验重复 5 次取平均, 足以说明明显趋势; 若进行更严格的性能研究, 应增加重复次数并报告标准差、中位数和置信区间。

10 结论

本实验使用 Microsoft SEAL 4.3.2 的 CKKS 方案, 实现了一个单输入单输出的密文卷积。程序将 4×4 输入按行打包到一个密文中, 通过密文旋转、明文掩码乘法、密文加法和重缩放得到 2×2 的加密输出。

作业 5 采用直接旋转累加法, 需要 8 次非零密文旋转。固定样例解密结果与明文结果一致, 最大绝对误差为 8.789110×10^{-6} 。作业 6 在完全相同的卷积任务上采用 BSGS 策略, 通过 2 次水平 Baby-Step 旋转和 2 次垂直 Giant-Step 旋转, 将旋转次数降低到 4 次。BSGS 最大绝对误差为 4.853192×10^{-7} , 5 组随机测试全部通过。

性能测试中, 直接法平均耗时为 40.883 ms, BSGS 法平均耗时为 18.171 ms, 获得 2.250 倍加速, 说明减少高成本密文旋转能够显著提高同态卷积性能。

理论分析从两个层次证明了最小旋转次数。首先, 在两级 BSGS 模型中, 3 次旋转最多覆盖 6 种组合, 而 4 次可覆盖 9 种组合。其次, 在更一般的单密文旋转-明文掩码-加法模型中, r 次旋转最多产生 2^r 种输入偏移; 由于稠密 3×3 卷积需要 9 种偏移,

至少需要 $\lceil \log_2 9 \rceil = 4$ 次旋转。因此，BSGS 实现达到当前实验模型下的理论下界。

需要强调的是，BSGS 不是唯一可能达到 4 次的构造；其他旋转树或分治方法可能与其持平，但不能在不改变模型的情况下低于 4 次。若采用多个输入密文、冗余打包、稀疏或可分离卷积核、系数编码等方法，服务器侧旋转次数可能进一步减少，但会引入通信、存储、预处理或适用范围方面的代价。

综上，本实验同时完成了密文卷积正确性验证、旋转次数优化、理论下界分析和性能对照，说明在全同态加密程序设计中，应重点识别旋转和密钥切换等高成本操作，并通过合理的数据布局与计算顺序进行复用。

参考资料

- [1] Microsoft SEAL 4.3.2 源代码、构建配置与示例程序。
- [2] 源代码：main.cpp。
- [3] 课程课件：《对称密码的软件实现与优化》。
- [4] 课程课件：全同态加密相关教学材料。
- [5] 本实验运行日志：CKKS 固定样例、随机测试和性能测试结果。

A 核心常量和数据布局

```
1 constexpr std::size_t kInputRows = 4;
2 constexpr std::size_t kInputCols = 4;
3 constexpr std::size_t kKernelRows = 3;
4 constexpr std::size_t kKernelCols = 3;
5 constexpr std::size_t kOutputRows = 2;
6 constexpr std::size_t kOutputCols = 2;
7 constexpr double kTolerance = 1e-4;
8
9 const std::array<int, 9> kOffsets{
10     0, 1, 2, 4, 5, 6, 8, 9, 10};
11 const std::array<std::size_t, 4> kOutputSlots{
12     0, 1, 4, 5};
```

Listing 21 输入、卷积核、输出规模及槽位布局